Lecture 9 — Stream Processing

CS4225 / CS5425 · Big Data Systems for Data Science · National University of Singapore

1 What is Stream Processing?

Core insight: In many real-world settings, data is not sitting still — it arrives continuously and endlessly over time. Stream processing is the discipline of computing results *as data arrives*, rather than waiting to collect the whole dataset first.

Traditional **batch processing** (MapReduce, SQL queries on files) operates on a fixed, complete dataset. You load all the data, run the job, get the answer. This works well for historical analysis, but it falls apart when you need answers within seconds or milliseconds of an event happening.

Streaming (online) processing treats data as an unbounded, ever-arriving sequence of records called **tuples**. Because the stream never ends, you cannot store all of it in memory — instead, you process each element as it arrives and maintain only the *state* you need to answer your queries.

Sensor Data

Industrial sensors, weather stations, medical monitors — all emit readings continuously. Anomaly detection must be instant.

Online User Activity

Clicks, page views, purchases — e-commerce and social platforms need real-time personalisation and fraud checks.

Financial Transactions

Credit card fraud detection and stock trade analysis must process millions of transactions per second with sub-second latency.

Stateful stream processing goes beyond simple record-at-a-time transformations. It stores and accesses intermediate data (state) — in program variables, local files, or external databases — to compute aggregates, joins, and pattern detections across many events. Exactly-once processing guarantees ensure results are always correct.

2 Why Does It Matter? — Batch vs. Stream

Batch systems introduce **pipeline latency**: data must first be written to storage (e.g. HDFS), then an ETL job runs, then the result is served. A traditional data warehouse pipeline might take minutes to hours. A streaming analytics application replaces that entire chain with a single stream processor that ingests events, maintains state, and updates results continuously — reducing the time from event to insight from hours to **milliseconds**.

Batch (Traditional)

Collect all data → store → ETL → warehouse → query.
Latency: minutes to hours.

→

Streaming

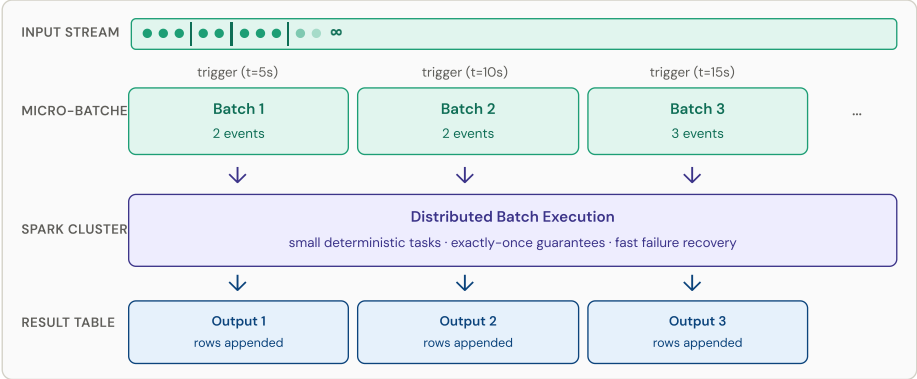
Event arrives → processed immediately → result updated.
Latency: milliseconds to seconds.

Exam trap: "Stream processing replaces batch" is wrong. They are complementary. Streaming handles low-latency, real-time needs; batch handles large-scale historical analysis. Many architectures use both (Lambda / Kappa architectures).

3 Spark Structured Streaming — Micro-batch Model

Spark's approach to streaming is called **Structured Streaming**. Rather than processing each event individually, it uses a **micro-batch** model: incoming stream data is collected into small time-bounded chunks (micro-batches), and each micro-batch is processed as a regular Spark batch job.

Key philosophy: Writing a streaming pipeline should feel exactly like writing a batch pipeline. Spark achieves this by modelling the data stream as an **unbounded table** — new rows keep arriving and being appended to the bottom of the table. Queries written against this table run continuously.



Advantage **Fault Tolerance**

Because each micro-batch is a deterministic Spark job, failures can be recovered by simply re-running the failed micro-batch. This gives exactly-once end-to-end guarantees.

Disadvantage **Latency**

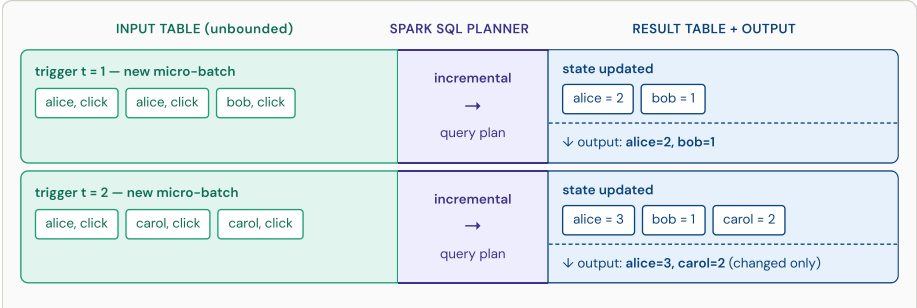
Micro-batching introduces latency of a few seconds. For most applications this is acceptable — many pipelines already incur delays elsewhere (network, database, etc.).

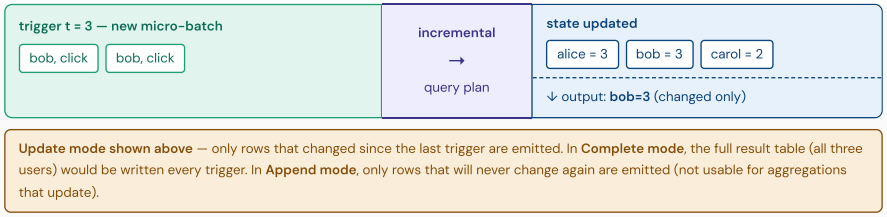
Example — how incremental execution works in practice

Suppose you write this query on an unbounded stream of click events. To you it looks exactly like a batch query:

```
SELECT user, COUNT(*) AS clicks
FROM click_events
GROUP BY user
```

Spark's SQL Planner converts this into an **incremental execution plan**. It identifies what state to keep (the per-user count) and how to update it with only the new events that arrived since the last trigger — it never recomputes from scratch.





Update mode shown above — only rows that changed since the last trigger are emitted. In **Complete mode**, the full result table (all three users) would be written every trigger. In **Append mode**, only rows that will never change again are emitted (not usable for aggregations that update).

Trigger	New events arriving	State after update	Output emitted (update mode)
t = 1	alice×2, bob×1	alice=2, bob=1	alice=2, bob=1
t = 2	alice×1, carol×3	alice=3, bob=1, carol=3	only new/changed rows since t=1
t = 3	bob×2	alice=3, bob=3, carol=3	only new/changed rows since t=2

Key formula: Result(t+1) = Result(t) + changes from new micro-batch data — Spark never reprocesses old events already captured in state.

One sentence to remember: Spark Structured Streaming = batch query + incremental state updates over time. You write once as if it were batch; Spark handles the streaming mechanics invisibly.

4 Output Modes

When Spark writes results of a streaming query to an external sink (database, file, console), it must know *how much* to write after each trigger. There are three modes:

Append Mode

Only **newly added rows** since the last trigger are written. Use when old rows will never change — e.g. raw event logs. Cannot be used with aggregations that may update old results.

Update Mode

Only **rows that changed** since the last trigger are written. Suitable for aggregations with a sink that supports in-place updates (e.g. MySQL table). Efficient: only sends deltas.

Complete Mode

The **entire result table** is rewritten after every trigger. Use for aggregations that produce a small result that must always be fully materialised (e.g. live leaderboard).

Exam trap: Append mode cannot be used with aggregations that can retroactively change (e.g. `count()` over an unbounded group). Use Update or Complete in those cases.

5 Five Steps to Define a Streaming Query

1. **Define input sources** — where is the stream coming from? (e.g. Kafka topic, file directory, socket). Use `readStream` with the appropriate format and options.
2. **Transform data** — apply stateless or stateful transformations (select, filter, groupBy, window aggregations). Same DataFrame/SQL API as batch.
3. **Define output sink and output mode** — where to write results (console, file, Kafka, database) and which output mode (Append / Update / Complete).
4. **Specify processing details** — (a) **Trigger interval:** how often to process new data (e.g. every 5 seconds, or as fast as possible); (b) **Checkpoint location:** a path (e.g. HDFS) where Spark stores query progress so it can recover from failures.
5. **Start the query** — call `.start()` on the `writeStream` object. The query runs continuously in the background.

6 Stateless vs. Stateful Transformations

Not all stream operations are equal. The critical distinction is whether a transformation needs to *remember* anything from previous events.

Stateless Transformations

Process each row in isolation — no memory of past rows needed. Examples:

- `select()`, `map()` — project columns
- `filter()`, `where()` — discard rows
- `explode()`, `flatMap()` — expand arrays

Think of these like a **calculator**: each input produces an output with no memory of what came before.

Stateful Transformations

Accumulate results across many events. Think of these like a **spreadsheet**: cells remember previous values and update when new data arrives.

The computation is: **output = f(current input, previous state)**

Each new event reads the current state, updates it, and emits a result. Example: `groupBy().count()` — each arriving event increments a per-key counter stored as state.

Concrete example — counting clicks per user:

```
Event 1: ("user1", click)  →  state: { user1: 1 }  →  emit user1=1
Event 2: ("user1", click)  →  state: { user1: 2 }  →  emit user1=2
Event 3: ("user2", click)  →  state: { user1: 2, user2: 1 }  →  emit user2=1
```

Where is state stored? In distributed stream processors, state can live in different places — each with a latency/durability trade-off:

In-memory variables

Fastest reads/writes. Volatile — lost if the node crashes. Used for hot state in Flink executors and Spark executors.

Local disk

More durable than memory. Still tied to one machine — lost if the machine is replaced. Used by embedded stores (e.g. RocksDB in Kafka Streams / Flink).

Distributed filesystem

Durable and replicated (e.g. HDFS, S3). Used for checkpoints — not for live reads/writes (too slow for per-event access).

External database

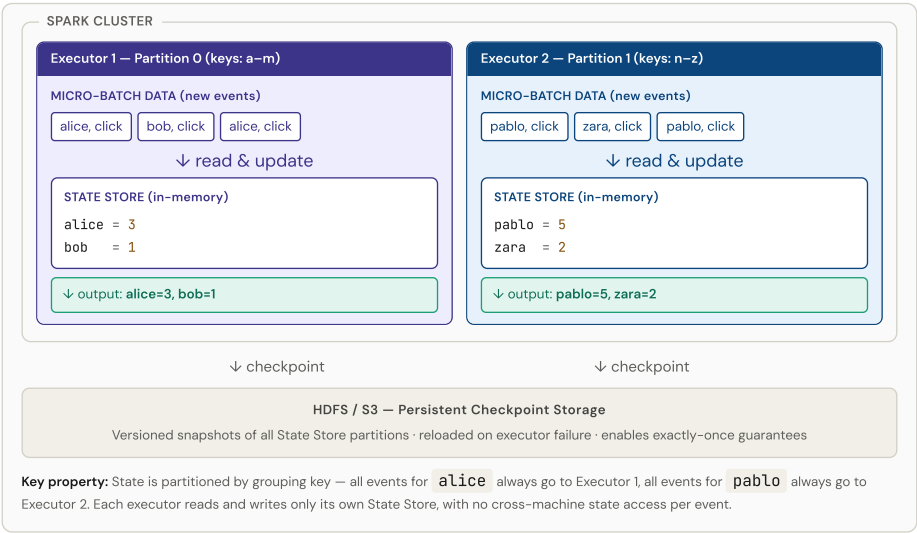
e.g. Redis, Cassandra. Sharable across systems. Slowest due to network calls — use only when state must be accessed outside the stream processor.

Exactly-once matters here: In a distributed system, failures cause replays. Without protection: (1) process event → update state; (2) crash before checkpoint; (3) event is replayed → state updated *twice*. Exactly-once guarantees prevent this via checkpointing + deterministic replay, ensuring each event affects the state once and only once.

Supported aggregation functions in stateful streaming: `sum()`, `mean()`, `stddev()`, `countDistinct()`, `collect_set()`, `approx_count_distinct()`, and more — all the same functions available in batch DataFrames.

Distributed state management in Structured Streaming

State cannot live on one machine at scale. Spark partitions state by grouping key across executors — each executor owns an exclusive subset of keys and holds a local **State Store** (an in-memory key-value map) for those keys. After each micro-batch, every State Store is checkpointed to HDFS/S3 so it can be restored on failure.



Key property: State is partitioned by grouping key — all events for `alice` always go to Executor 1, all events for `pablo` always go to Executor 2. Each executor reads and writes only its own State Store, with no cross-machine state access per event.

7 Time Semantics — Processing Time vs. Event Time

Processing Time

The wall-clock time *at the Spark cluster* when the record is processed. Simple to implement but **not deterministic** — if you replay the same stream faster, you get different window groupings. Affected by network delays.

Event Time

The timestamp *embedded in the event itself* — when the event actually happened (e.g. the time a sensor reading was taken, or the time a transaction was made). **Deterministic** — replaying the same stream always yields the same results, regardless of processing speed.

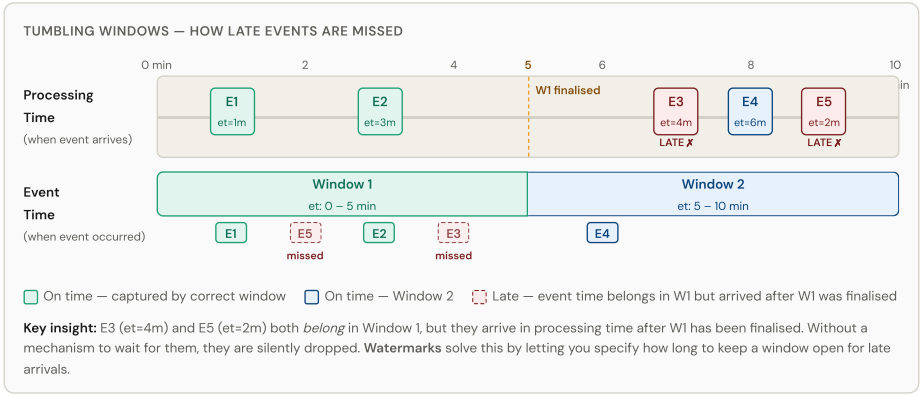
Analogy: Imagine you are on a train in a tunnel and send a message at 3 pm, but your phone only connects and sends it at 3:05 pm. The *event time* is 3:00 pm (when you composed it); the *processing time* is 3:05 pm (when the server received it).

> **Why event time matters:** Operations based on event time are predictable and deterministic. A window from 3:00–3:05 will always contain the same events no matter how fast or slow the stream is processed, or how delayed delivery was. Processing-time windows do not have this guarantee.

The query below groups sensor readings into 5-minute **tumbling windows** based on event time and counts how many readings fall in each window per sensor:

```
sensorReadings.groupBy('sensorId', window("eventTime", "5 minutes")).count()
```

The problem: sensors can be offline, networks lag, and events do not always arrive in the order they occurred. The diagram below shows how events with event times inside Window 1 can **arrive late in processing time** — after Window 1 has already been finalised — and therefore be silently missed.

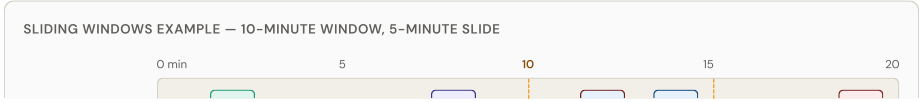


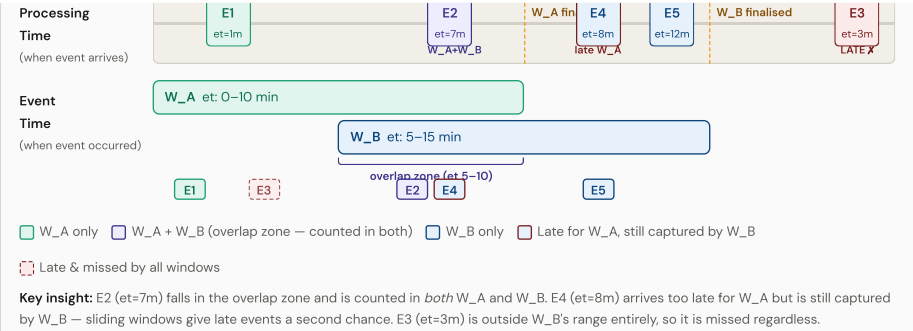
Key insight: E3 (et=4m) and E5 (et=2m) both *belong* in Window 1, but they arrive in processing time after W1 has been finalised. Without a mechanism to wait for them, they are silently dropped. **Watermarks** solve this by letting you specify how long to keep a window open for late arrivals.

Contrast with a **sliding window** — the third argument sets a slide interval smaller than the window duration, so consecutive windows overlap:

```
sensorReadings.groupBy("sensorId", window("eventTime", "10 minutes", "5 minutes")).count()
```

With a 10-minute window and 5-minute slide, a new window opens every 5 minutes. Each event with event time in the **overlap zone (et 5–10 min)** is counted in *two* windows simultaneously. The late-event problem still applies — but an event late for one window can still be on time for another.





The Late Data Problem: If we use event time, how long do we wait before closing a window? Events can arrive out of order due to network lag, retries, or mobile devices going offline. We need a mechanism to say "I am reasonably sure I have seen all events up to time T" — this is a **watermark**.

> **Watermark:** A threshold that tells the system: "No more events with event-time ≤ T should be expected." Once the watermark passes a window's end time, that window is finalised and its state can be discarded.

In Spark Structured Streaming, a watermark is specified as a **delay tolerance** on an event-time column:

```
# Tolerate up to 10 minutes of late arrival
streamDF.withWatermark("eventTime", "10 minutes")
        .groupBy(window("eventTime", "5 minutes"), "userId")
        .count()
```

Spark keeps the window state alive for up to 10 minutes beyond the window end. Events arriving more than 10 minutes late are silently dropped.

> **Exam tip:** Watermarks serve a dual purpose in Spark: (1) they define when late data is accepted/dropped, and (2) they allow Spark to clean up old window state from memory — without watermarks, stateful aggregations would accumulate state forever and eventually run out of memory.

Extra Performance Tuning — Spark Structured Streaming

Beyond tuning the Spark SQL engine (e.g. Catalyst optimiser settings), streaming workloads have unique operational demands that require additional consideration. These apply specifically when running 24/7 continuous streaming pipelines.

Cluster Resource Provisioning

Streaming jobs run **continuously, 24/7** — unlike batch jobs that spin up and terminate. This means you must provision enough executors and memory to handle *peak throughput* at all times, not just average load. Under-provisioning leads to backpressure: incoming data accumulates faster than it can be processed, causing ever-growing queues and eventual failure.

Number of Shuffle Partitions

In batch queries, the default shuffle partition count (e.g. 200) is usually fine because each job processes a large dataset. In streaming, each micro-batch is small — using 200 partitions for a tiny batch creates massive scheduler overhead. The partition count should be set **much lower** (e.g. equal to the number of cores) to avoid this per-micro-batch overhead.

```
spark.conf.set("spark.sql.shuffle.partitions", "8")
```

Source Rate Limits

A burst of incoming data (e.g. a spike in Kafka messages) could cause one micro-batch to be enormous, destabilising the pipeline. Setting a **max records per trigger** limit on the source caps ingestion rate per batch, smoothing out spikes and preventing out-of-memory errors.

```
df.readStream.option("maxOffsetsPerTrigger", "10000")
```

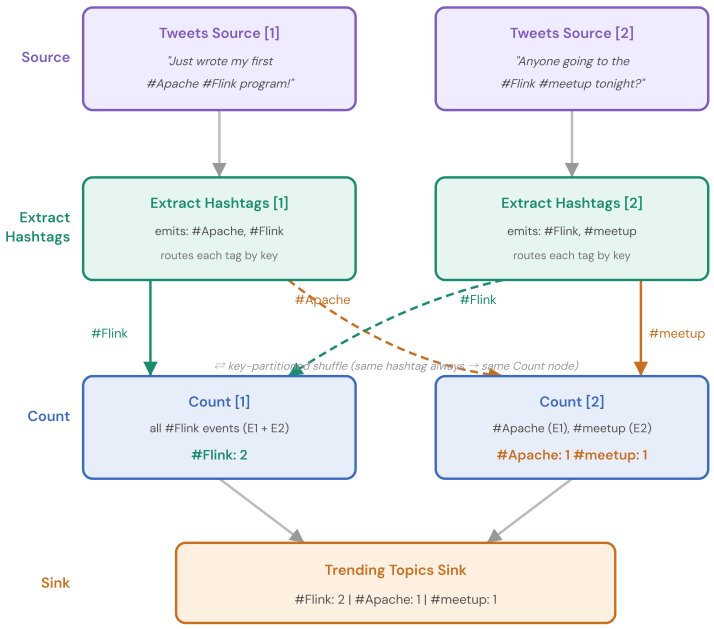
Multiple Streaming Queries in One App

You can run **multiple streaming queries concurrently within a single Spark application**, sharing the same SparkSession and cluster resources. This is useful for processing different topics or applying different transformations to the same stream in parallel. However, resource contention between queries must be managed — use Spark's fair scheduler to ensure each query gets a fair share of executor threads.

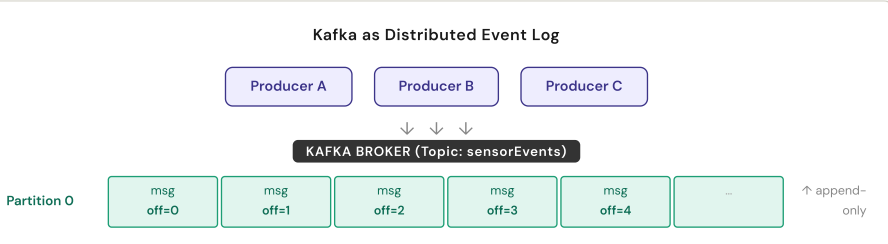
8 Apache Flink — Architecture & Dataflow Model

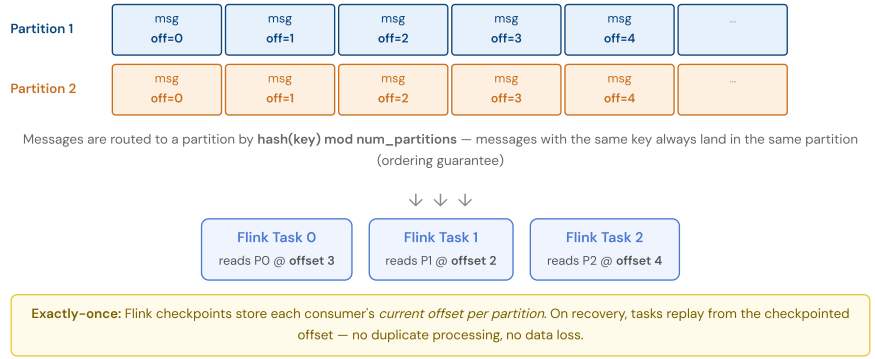
Flink is a **true continuous stream processing engine** — unlike Spark's micro-batch approach, Flink processes each event individually as it arrives (event-driven streaing application), achieving **millisecond** latency. Flink also supports batch processing, but streaming is its native model.

> **Dataflow model:** A Flink application is expressed as a **dataflow graph** — a directed acyclic graph (DAG) where **nodes are operators** (transformations) and **edges are data dependencies** (streams of records flowing between operators). This logical graph is compiled into a physical execution plan where each operator runs as parallel tasks.



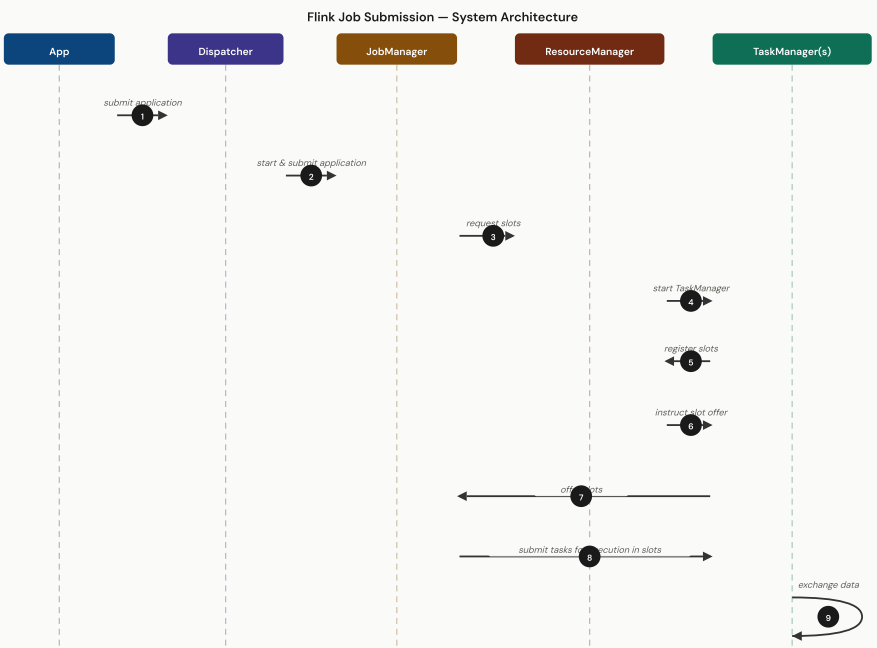
Kafka as Event Log: Flink (and Spark Streaming) commonly ingest data from **Apache Kafka**, a distributed message queue. Kafka partitions messages by a hash key across multiple partitions. Consumers read from a partition at a specific *offset* — a sequential position number. This decouples producers from consumers, supports asynchronous data transfer, and enables exactly-once semantics when combined with Flink's checkpointing.





> **Exactly-once:** Flink checkpoints store each consumer's *current offset per partition*. On recovery, tasks replay from the checkpointed offset — no duplicate processing, no data loss.

System Architecture: Flink runs as a distributed system with two main components:



TaskManager

Worker nodes. Each TaskManager has a fixed number of **processing slots** to control the number of tasks it is able to concurrently execute. Each slot can run one parallel task of each operator of the application. A TaskManager can run tasks of the same operator (data parallelism), different operators (task parallelism), or from a different application (job parallelism).

> **Data Transfer:** Tasks of a running application are continuously exchanging data. TaskManagers collect outgoing data from sending tasks into **buffers** before shipping them to receiving tasks over the network. This batching improves throughput without adding significant latency.

9 Flink — Event-Time Processing & Watermarks

Like Spark, Flink supports both processing time and event time. For event-time processing, every record must carry an **event timestamp**. But Flink's implementation of watermarks is fundamentally different from Spark's.

> **Key difference:** In Spark, a watermark is a user-specified configuration (a delay tolerance on a column). In Flink, a **watermark is an actual special record** — it flows *inline within the data stream* alongside regular event records, carrying a timestamp value of type `Long`.

A Watermark with timestamp T tells downstream operators: *"I am reasonably sure no more events with timestamp ≤ T will arrive."* When a window operator receives a watermark of time T, it **triggers the computation** of all windows whose end time is ≤ T.

Watermark Strategies in Flink:

Strategy	When to Use	How it Works
Monotonous Timestamps	Data arrives in perfect order (rare in practice)	Watermark = Maximum event timestamp seen so far. Assumes no out-of-order events.
Bounded Out-of-Orderness	Real-world streams with known maximum delay L	Watermark = Max timestamp observed - L. Waits up to L seconds for late events before closing windows.
Custom Strategy	Complex or irregular data patterns	You implement your own logic for when and how the watermark advances. Full flexibility.

Handling Late Events in Flink: Even with a delay buffer L, some events arrive even later. Flink offers three strategies:

Default: Discard

Once the watermark passes the window's end time, any event arriving after that is simply **dropped**. Simple and memory-efficient, but may miss some data.

Allowed Lateness

Keep the window state alive for an extra duration beyond the watermark. If a late event arrives within this extra window, the window is **re-fired** to update and emit a corrected result.

Side Outputs

Very late events are diverted to a **separate side stream** for manual inspection, logging, or separate processing logic. The "safety net" — nothing is lost.

10 Flink — State Management & Checkpointing

Every stateful operator instance in Flink maintains **local state in memory** — this enables fast reads and writes for each incoming record. But TaskManager memory is volatile — if a process crashes, all in-memory state is lost. Flink solves this with **checkpointing**.

> **Checkpoint:** A consistent, distributed snapshot of all operator states written to remote persistent storage (e.g. HDFS, S3). Checkpoints are triggered *automatically* and are used for **failure recovery**. If a task fails, Flink restarts the application from the latest checkpoint.

Flink's Checkpointing Algorithm is based on the **Chandy–Lamport distributed snapshot algorithm**. Its key property: it does *not* pause the whole application. Instead it uses a clever mechanism called **checkpoint barriers**:

1. The **JobManager** initiates a checkpoint by sending a message to all source operators.
2. **Source operators** checkpoint their own state (i.e. the last thing they processed) and inject a special **checkpoint barrier** record into each of their output streams. The barrier carries a checkpoint ID.

```
source 1 & 2 have streams to 2 tasks (sum_even and sum_odd) each.
both sources inject 2 barriers each = 4 barriers total.
last number before checkpoint: source 1 - 3, source 2 - 7. these are checkpointed.
```

3. Downstream tasks wait to receive a barrier on *each* input partition. Records arriving *after* the barrier on one partition but *before* the barrier on another are **buffered**; all other records continue to be processed normally.

```
sum_even receives barrier from source 1 but not from source 2 yet.
sum_even will buffer records coming from source 1 until barrier from source 2 arrives.
```

4. Once a task receives barriers from *all* input partitions, it **checkpoints its own state** to remote storage, then forwards the checkpoint barrier downstream.

```
current state in sum_even: 8
source 2 gives 2, then the barrier
updated state = 8 + 2 = 10 ← this is checkpointed.
```

5. Tasks **resume normal processing** immediately after forwarding the barrier — they do not wait for the checkpoint to complete.
6. When all operators (including sinks) acknowledge receipt of the checkpoint barrier to the JobManager, the **checkpoint is complete**.

> **Why this is efficient:** The Chandy-Lamport approach is **asynchronous** — some tasks are checkpointing while others continue processing data. There is no global pause (stop-the-world). This is why Flink can achieve millisecond latencies even with checkpointing enabled. Contrast with Spark: each micro-batch is implicitly checkpointed *synchronously* at the end of every batch.

Failure Recovery from a Checkpoint:

1. Restart
Restart the entire application (all operators).

→

2. Reset State
Reload all stateful operators' state from the latest checkpoint.

→

3. Resume
Resume processing from the positions in the sources recorded in the checkpoint.

11 Flink — Savepoints

A **Savepoint** is similar to a checkpoint in that it captures the full state of a running Flink job — but it is **manually triggered by the operator** (not automatic), and is designed for **operational use cases** rather than failure recovery.

Checkpoint

Automatic, periodic. Used for failure recovery. May be overwritten by newer checkpoints. Tightly coupled to a specific job instance.

Savepoint

Manual, on-demand. Designed for planned operations. **Portable** — works across Flink versions and cluster configurations. Contains operator state + source offsets.

Use cases for Savepoints:

1. **Application Upgrades:** Take a savepoint → stop the job → deploy new code JAR → resume from the savepoint. Zero state loss.
2. **Scaling:** Changing parallelism (e.g. 10 workers → 50 workers) requires a restart from a savepoint so Flink can redistribute state partitions across the new worker count.
3. **Cluster Migration:** Move a running job from on-premise to the cloud — just point the new cluster at the savepoint file path.
4. **A/B Testing:** Fork two different versions of the application from the exact same savepoint to compare performance on identical starting state.

12 Spark vs. Flink — Comparison

Dimension	Spark Structured Streaming	Apache Flink
Processing Model	Micro-batch: collects events into small batches, processes each as a Spark job	True continuous streaming: processes each event individually as it arrives
Latency	Seconds (configurable micro-batch interval)	Milliseconds (sub-second)
Checkpointing	Synchronous — occurs at the end of every micro-batch. Simple but adds overhead per batch.	Asynchronous — Chandy-Lamport algorithm with checkpoint barriers. More efficient; does not pause processing.
Watermark Type	A *configuration*: user specifies a delay tolerance on an event-time column (`withWatermark`)	A *special record* that flows inline in the data stream, carrying a `Long` timestamp
Late Data Handling	Events past the watermark threshold are dropped	Three options: Discard, Allowed Lateness (re-fire window), Side Outputs
Programming Model	DataFrame/SQL API — same as batch. Easy to learn for Spark users.	DataStream API (low-level) or Table/SQL API. More control but steeper learning curve.
Exactly-Once	Supported via deterministic micro-batch re-execution	Supported via distributed checkpoints + transactional sinks
Savepoints	Not a native concept (checkpoints are per-query)	First-class feature for upgrades, scaling, and migration
Native Model	Batch-first; streaming added on top via micro-batching	Stream-first; batch is treated as a finite stream
Best For	Applications tolerating few-second latency; teams already using Spark; unified batch+stream pipelines	True real-time applications (fraud detection, trading); complex event processing; stateful streaming at scale

13 Exam-Style Q&A

Q: What is the difference between processing time and event time? Why does event time produce deterministic results?

> Processing time is the wall-clock time when an event is processed by the cluster. Event time is the timestamp embedded in the event itself (when it actually occurred). Event time is deterministic because the same event will always fall into the same window regardless of how fast or slow processing is, or how delayed delivery was. Processing time results can change if you replay data at a different speed.

Q: What is a watermark and what two problems does it solve in Spark Structured Streaming?

> A watermark is a threshold defined as a maximum tolerated delay on an event-time column (e.g., `withWatermark("eventTime", "10 minutes")`). It solves two problems: (1) it defines when late-arriving events are accepted vs. dropped — events more than 10 minutes late are discarded; (2) it allows Spark to clean up (garbage collect) old window state from executor memory, preventing unbounded memory growth in stateful aggregations.

Q: How does Flink's checkpointing algorithm avoid pausing the whole application?

> Flink uses the Chandy-Lamport algorithm with *checkpoint barriers*. The JobManager triggers a checkpoint by injecting a barrier record into source streams. As the barrier flows downstream, each operator checkpoints its state upon receiving barriers from all inputs, then forwards the barrier. Crucially, tasks that have already forwarded the barrier continue processing new data while others are still checkpointing. This async, decoupled approach means no global pause — some tasks are always processing.

Q: What are the three output modes in Spark Structured Streaming? When can Append mode NOT be used?

> Append: write only new rows added since last trigger. Update: write only changed rows. Complete: rewrite entire result table. Append mode cannot be used when existing rows can change — for example, `groupBy().count()` can retroactively update counts as new events arrive in a window, violating the append-only constraint. Use Update or Complete for such aggregations.

Q: What is a Flink Savepoint, and how does it differ from a Checkpoint?

> A Checkpoint is automatic and used for failure recovery — it may be overwritten. A Savepoint is manually triggered by the operator and designed for planned operations (upgrades, scaling, migration, A/B testing). Savepoints are portable across Flink versions and cluster configurations, while checkpoints are tightly coupled to a specific job instance. Both capture the full operator state and source offsets.

Q: What are the three ways Flink can handle events that arrive after the watermark has passed?

> (1) **Default (Discard):** late events are simply dropped once the watermark passes the window end time. (2) **Allowed Lateness:** keep the window state alive for an extra duration; if a late event arrives within this window it re-fires the window with updated results. (3) **Side Outputs:** divert very late events to a separate side stream for manual inspection or separate processing — nothing is lost.

Q: Explain the key architectural difference between Spark Structured Streaming and Apache Flink regarding how they achieve exactly-once guarantees.

> Spark achieves exactly-once via deterministic micro-batch re-execution — if a batch fails, it is simply re-run from the same source offsets. Flink achieves exactly-once via distributed asynchronous checkpointing (Chandy-Lamport) combined with transactional sink connectors — when a checkpoint completes, Flink can guarantee that all records included in that checkpoint are committed exactly once to the output. Spark's approach is simpler but adds per-batch latency; Flink's is more complex but allows millisecond-latency exactly-once.